COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE 010/Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	September 10, 2026	**DATE SUBMITTED:**	September 14, 2026
NAME:	Jaime D.C. Godino Jr.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 8.1 **Sorting Algorithms Pt2**

1. PROCEDURE OUTPUT

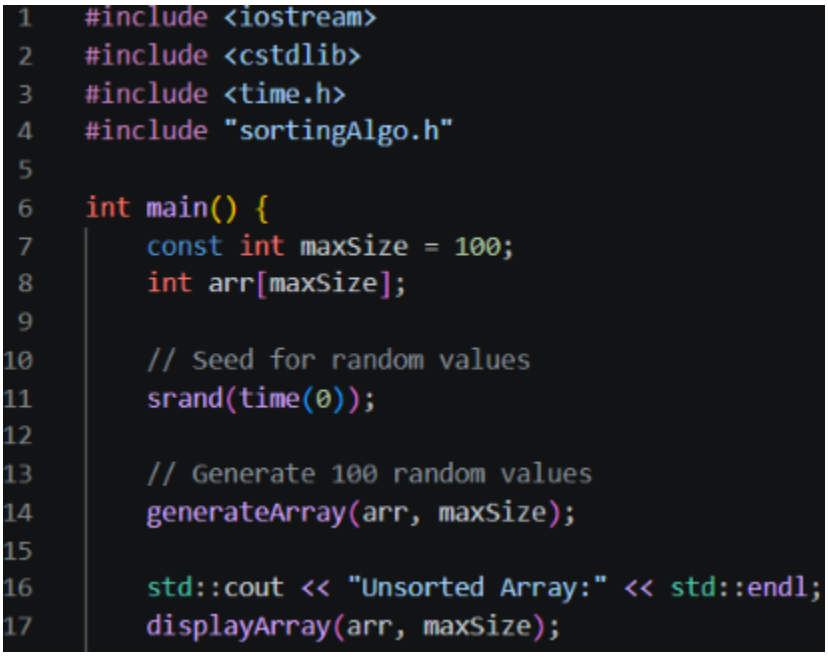
ILO A: Create C++ code for implementation of shell, merge, and quick sort.

Code + Console Screenshot

sortingAlgo.h

```
4   //generate an array of elements containing 100
5   template <typename T>
6   void generateArray(T arr[], int size) {
7       for (int i = 0; i < size; i++) {
8           arr[i] = rand() % 100 + 1;
9       }
10  }
11
12  //display all elements inside the array
13  template <typename T>
14  void displayArray(T arr[], int size) {
15      for (int i = 0; i < size; i++) {
16          std::cout << arr[i] << " ";
17
18          if ((i + 1) % 10 == 0) {
19              std::cout << std::endl;
20          }
21      }
22  }
```

main.cpp



```cpp
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "sortingAlgo.h"

int main() {
    const int maxSize = 100;
    int arr[maxSize];

    // Seed for random values
    srand(time(0));

    // Generate 100 random values
    generateArray(arr, maxSize);

    std::cout << "Unsorted Array:" << std::endl;
    displayArray(arr, maxSize);
```

output

```
Unsorted Array:
87 53 24 65 46 28 34 85 24 26
96 4 67 31 86 91 12 27 3 23
59 3 8 30 32 39 74 97 93 24
51 36 82 64 31 61 16 82 69 9
21 80 75 67 71 65 20 58 79 76
12 13 59 42 41 1 31 12 73 35
33 81 38 91 32 1 13 97 34 26
58 77 8 61 40 60 77 64 28 96
93 95 2 80 93 69 32 21 76 32
29 64 64 59 65 38 57 31 32 16
```

Observations	These lines of code generate 100 random integers between 1 and 100 using time.h. The same unsorted array should be reused across all algorithms for the same comparisons. This makes the runtime and comparison counts fast for the algorithms.

Table 8-1. Array of Values for Sort Algorithm Testing

<table>
<tr>
<td>Code + Console Screenshot</td>
<td>

sortingAlgo.h

```
24  template <typename T>
25  void shellSort(T arr[], int maxSize) {
26      for (int interval = maxSize / 2; interval > 0; interval /= 2) {
27          for (int i = interval; i < maxSize; i += 1) {
28              T temp = arr[i];
29
30              int j;
31              for (j = i; j >= interval && arr[j - interval] > temp; j -= interval) {
32                  arr[j] = arr[j - interval];
33              }
34              arr[j] = temp;
35          }
36      }
37  }
```

main.cpp

```
19      std::cout << "\nShell Sort:" << std::endl;
20      shellSort(arr,maxSize);
21      displayArray(arr, maxSize);
```

output

```
Shell Sort:
1 1 2 3 3 4 8 8 9 12
12 12 13 13 16 16 20 21 21 23
24 24 24 26 26 27 28 28 29 30
31 31 31 31 32 32 32 32 32 33
34 34 35 36 38 38 39 40 41 42
46 51 53 57 58 58 59 59 59 60
61 61 64 64 64 64 65 65 65 67
67 69 69 71 73 74 75 76 76 77
77 79 80 80 81 82 82 85 86 87
91 91 93 93 93 95 96 96 97 97
```

</td>
</tr>
<tr>
<td>Observations</td>
<td>This compares elements separated by a gap and moves them into their correct position. For an array of 100 elements, it performs 6 passes using gaps of 50, 25, 12, 6, 3, and 1. After the final pass, the array is sorted in ascending order. Its time complexity depends on the gap sequence, with a best-case complexity of O(nlogn) and worst-case complexity of O(n^2) for this implementation.</td>
</tr>
</table>

Table 8-2. Shell Sort Technique

Code + Console Screenshot	**sortingAlgo.h**
	`template <typename T>`
	`void merge(T arr[], int left, int middle, int right) {`
	`    int leftSize = middle - left + 1;`
	`    int rightSize = right - middle;`
	
	`    T* leftArr = new T[leftSize];`
	`    T* rightArr = new T[rightSize];`
	
	`    for (int i = 0; i < leftSize; i++)`
	`        leftArr[i] = arr[left + i];`
	`    for (int j = 0; j < rightSize; j++)`
	`        rightArr[j] = arr[middle + 1 + j];`
	
	`    int i = 0; //pointer into leftArr`
	`    int j = 0; //pointer into rightArr`
	`    int k = left; //pointer into arr (merged output)`
	
	`    while (i < leftSize && j < rightSize) {`
	`        if (leftArr[i] <= rightArr[j]){`
	`            arr[k] = leftArr[i];`
	`            i++;`
	`        } else {`
	`            arr[k] = rightArr[j];`
	`            j++;`
	`        }`
	`        k++;`
	`    }`

```
67      //copy any remaining elements
68      while (i < leftSize) {
69          arr[k] = leftArr[i];
70          i++;
71          k++;
72      }
73      while (j < rightSize) {
74          arr[k] = rightArr[j];
75          j++;
76          k++;
77      }
78
79      delete[] leftArr;
80      delete[] rightArr;
81  }
82
83  template <typename T>
84  void mergeSortHelper(T arr[], int left, int right) {
85      if (left >= right) {
86          return;
87      }
88      int middle = (left + right) / 2;
89
90      mergeSortHelper(arr, left, middle); //sort first half
91      mergeSortHelper(arr, middle + 1, right); //sort second half
92      merge(arr, left, middle, right); //merge sorted halves
93  }
94
```

```
95  template <typename T>
96  void mergeSort (T arr[], int maxSize) {
97      if (maxSize <= 1) return;
98      mergeSortHelper(arr, 0, maxSize - 1);
```

main.cpp

```
23      std::cout << "\nMerge Sort:" << std::endl;
24      mergeSort(arr, maxSize);
25      displayArray(arr, maxSize);
```

output

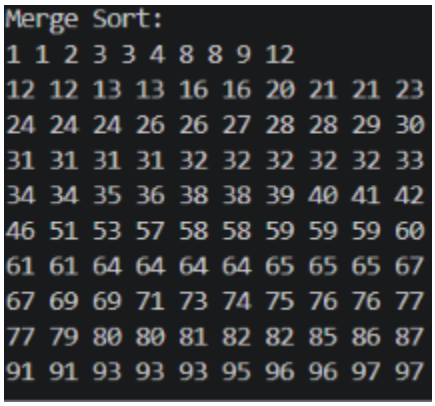
	
Observations	This recursively divides the array into smaller halves, sorting them, and merging them back together in ascending order. For the given 100-element array, it performs 99 merge operations across 7 levels of merging. After all the merging is completed, the array is sorted. Its best, average, and worst-case time complexity is O(nlogn).

Table 8-3. Merge Sort Algorithm

Code + Console Screenshot	**sortingAlgo.h**

```cpp
template <typename T>
int partition(T arr[], int low, int high) {
    T pivot = arr[high]; //choose last element as pivot
    int i = low - 1; //index of the last element <= pivot

    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            std::swap(arr[i], arr[j]);
        }
    }

    std::swap(arr[i + 1], arr[high]);; //place pivot in its correct position
    return i + 1; //return pivot's final index
}

template <typename T>
void quickSortHelper(T arr[], int low, int high) {
    if (low < high) {
        int pivot = partition(arr, low, high);

        quickSortHelper(arr, low, pivot - 1); //sort left of pivot
        quickSortHelper(arr, pivot + 1,  high); //sort right of pivot
    }
}

template <typename T>
void quickSort(T arr[], int maxSize) {
    if (maxSize <= 1) return;
    quickSortHelper(arr, 0, maxSize - 1);
}
```

main.cpp

```cpp
    std::cout << "\nQuick Sort:" << std::endl;
    quickSort(arr, maxSize);
    displayArray(arr, maxSize);
```

output

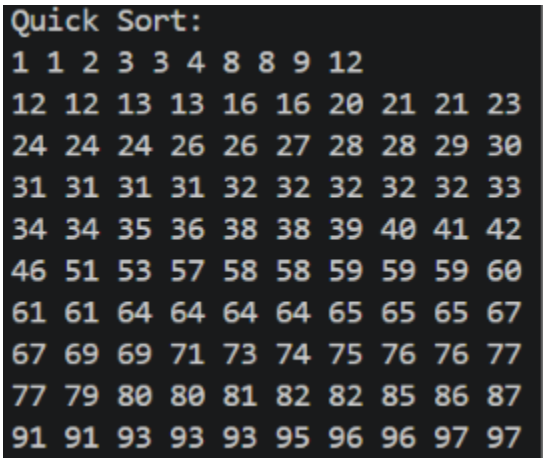
	
Observations	This works by selecting the last element as the pivot, partitioning the array around it, and recursively sorting the left and right portions. For the given array, the number of partitioning operations depends on the pivot positions, and the algorithm continues until all elements are sorted. Its average time complexity is O(nlogn), while its worst-case time complexity is $O(n^2)$ when partitions are unbalanced.

Table 8-4. Quick Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

- Yes, we can use other sorting algorithms to sort the left and right sublists created by the partition methon in quick sort. For instance, we can use insertion sort the sublist after partitioning. Given the array from geeks for geeks, {5, 6, 11, 12, 13}, we can choose 11 as the pivot, resulting in a left sublist of {5, 6} and a right sublist of {12, 13}. After applying insertion sort to both sublists, the elements remain in ascending order, and combining them with the pivot produces the final sorted array {5, 6, 11, 12, 13}. This demonstrates that other sorting algorithms can be used within quick sort to sort its sublists.

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

- For the given array of 15 elements, quick sort is a good option because it performs efficiently on large datasets. Merge sort and quick sort can achieve an average time complexity of O(nlogn) because both use the divide-and-conquer approach, which breaks the array into smaller subproblems. Merge sort divides the array into halves and merges them in sorted order, while quick sort partitions the array around a pivot and recursively sorts the sublists. Overall, both algorithms require approximately O(n) operations at each of the logn levels when the partitions are balanced, resulting in O(nlogn) time complexity.

Conclusion

In this activity, I understood how shell sort uses gaps, merge sort divides and merges subarrays, and quick sort partitions an array around a pivot. I also learned about their time complexities and how the choice of sorting algorithm affects the performance of a program. Overall, I think I performed well in this activity because I was able to understand the fundamentals of the sorting algorithms. However, I still need to learn how to implement binary search and linear search using those sorting algorithms.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Grammarly and ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction and explanation of concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used Grammarly to fix and improve my grammar while using ChatGPT to further understand the concept while working on the programs in the activity**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.